

CSC301 - Design and Analysis of Algorithms

Lecture 3: Algorithm Correctness Fundamentals

Worked Examples + Textbook Exercises with Model Solutions

CDF focus	Preconditions, postconditions, partial correctness, termination, total correctness, systematic
Main lecture example	Maximum-finding in an array
Levitin references	3rd ed., Exercises 1.2-Q3, 1.2-Q4, 1.2-Q8
CLRS reference	3rd ed., Exercise 2.1-3 (optional bridge to Lecture 4)
Copyright note	Book exercise wording is paraphrased; exercise numbers are preserved.

Use this handout after the Lecture 3 slides. The worked examples deliberately move from simple branch-based algorithms to the array maximum problem. The CLRS loop-invariant exercise is included only as a preview; the full invariant proof method belongs to Lecture 4.

1. Mathematical Symbols Used in Correctness Statements

Symbol	Read as	Meaning in this lecture
$\forall$	for all / for every	A statement must hold for every element or every valid input
$\in$	is an element of	$m \in A$ means m occurs in the array A
$\notin$	is not an element of	$x \notin A$ means x does not occur in A
$\geq$	greater than or equal to	$m \geq x$ means m is at least as large as x
$\leq$	less than or equal to	$x \leq m$ means x is no larger than m
$\neq$	not equal to	$b \neq 0$ excludes division by zero
$\wedge$	and	Both conditions must be true
$\Rightarrow$	implies	If the left condition is true, the right condition follows

Core correctness pattern: valid inputs satisfy the precondition; if the algorithm terminates, the postcondition must hold. Total correctness additionally requires that termination itself is guaranteed.

2. Worked Example 1 - DIVIDE(a, b)

Algorithm

```
DIVIDE(a, b)
return a / b
```

Precondition

Mathematical: $b \neq 0$

English: the divisor must not be zero.

Postcondition

Mathematical: $\text{result} = a / b$

English: the returned value is the quotient of a divided by b.

Partial correctness

Assume the precondition $b \neq 0$ is true. The division operation is then defined. If the algorithm returns, the returned result is exactly a/b , so the postcondition holds.

Termination

There is one arithmetic operation and one return statement; there is no loop or recursion. Therefore the algorithm terminates.

Conclusion

Partial correctness + termination = total correctness for every input satisfying $b \neq 0$.

3. Worked Example 2 - MAX2(a, b)

Algorithm

```
MAX2(a, b)
if a ≥ b
    return a
else
    return b
```

Precondition

a and b belong to a domain in which the comparison $\geq$ is defined (for example, real numbers).

Postcondition

Let m be the returned value.

$m \in \{a, b\}$ - the result is one of the two inputs.

$m \geq a \wedge m \geq b$ - the result is at least as large as both inputs.

Together these conditions mean $m = \max(a, b)$.

Reason through both branches

Condition	What the algorithm returns	Why the postcondition holds
$a \geq b$	a	a is one of the inputs and is not smaller than b
$a < b$	b	b is one of the inputs and is larger than a

The algorithm also terminates after one comparison and one return, so it is totally correct for its stated input domain.

4. Worked Example 3 - ABS(x)

```
ABS(x)
if x ≥ 0
    return x
else
    return -x
```

Precondition: x is a real number.

Postcondition: $\text{result} \geq 0$ and $\text{result} = x$ or $\text{result} = -x$. In plain English, the returned value is the non-negative magnitude of x .

Worked cases

Case 1: $x \geq 0$. The algorithm returns x , which is already non-negative.

Case 2: $x < 0$. The algorithm returns $-x$, which is positive.

The algorithm has no loop and therefore terminates immediately.

5. Worked Example 4 - MAX(A) [Main Lecture Example]

```
MAX(A)
largest ← A[0]
for i ← 1 to length(A)-1
    if A[i] > largest
        largest ← A[i]
return largest
```

Preconditions

- A is a valid finite array.
- $|A| = n \geq 1$. The array is non-empty because $A[0]$ is accessed.
- The elements are mutually comparable. The operation $A[i] > \text{largest}$ must be defined.
- Indices 0 through $n-1$ are valid.

Postconditions

Let m be the returned value.

1. $m \in A$ - the result actually occurs in the input array.
2. $\forall x \in A, m \geq x$ - no element of A is larger than m .
3. $A_{\text{after}} = A_{\text{before}}$ - this version of MAX does not modify the input array.

Therefore, $m = \max(A)$.

6. Why MAX Needs Both Postconditions

Suppose $A = [3, 8, 5]$ and a faulty algorithm returns $m = 100$.

The statement $\forall x \in A, m \geq x$ is true, because 100 is at least as large as 3, 8, and 5. But $m \in A$ is false because 100 is not an array element. Therefore 100 is not the maximum element of A .

Correct postcondition: $m \in A$ and $\forall x \in A, m \geq x$.

Trace one valid instance

Step	Current value	largest after step
Initialize	$A[0] = 8$	8
Compare	3	8
Compare	12	12
Compare	5	12
Compare	10	12

This trace demonstrates one instance. It is useful for understanding behavior, but by itself it is not a proof for every valid array.

7. Counterexample - A MAX Algorithm That Looks Fine but Is Wrong

```
MAX-WRONG(A)
largest ← 0
for each x in A
    if x > largest
        largest ← x
return largest
```

For $A = [-8, -3, -12, -5]$, the algorithm returns 0. But $0 \notin A$ and the true maximum is -3.

Lesson: termination does not imply correctness. Hidden assumptions introduced by initialization can violate the postcondition.

8. Textbook Exercise - Levitin 1.2-Q3

Paraphrased task: given positive side lengths a , b and c of a triangle, Levitin presents three familiar area formulas and asks which formulas can themselves be regarded as algorithms when only a , b and c are supplied.

Model solution

(a) Heron's formula can be treated as an algorithm if square root is an available primitive operation. All quantities needed can be computed from a , b and c .

(b) $\frac{1}{2}bc \sin(A)$ is not, by itself, a complete algorithm from the stated inputs because angle A is not supplied; an additional procedure is needed to derive A from the side lengths.

(c) $\frac{1}{2}a h_a$ is also incomplete from the stated inputs because h_a is not given and no procedure for computing it is specified.

Correctness connection

This exercise is useful for Lecture 3 because it exposes a hidden specification issue: an expression can be mathematically valid yet fail to be a complete algorithm for the given input specification.

9. Textbook Exercise - Levitin 1.2-Q4

Paraphrased task: design pseudocode that returns the real roots of $ax^2 + bx + c = 0$ for arbitrary real coefficients a , b and c , including all special cases.

Model algorithm

```
QUADRATIC(a, b, c)
if a ≠ 0
    D ← b2 - 4ac
    if D > 0: return the two real roots
    else if D = 0: return the repeated real root
    else: return 'no real roots'
else
    if b ≠ 0: return -c/b
    else if c = 0: return 'all real numbers'
    else: return 'no solution'
```

Correctness specification

Precondition: a , b and c are real numbers.

Postcondition: the output correctly classifies the equation and returns exactly its real solution set.

The key correctness lesson is that $a = 0$ cannot be ignored. If $a = 0$, the problem is no longer a quadratic equation; it becomes linear or degenerate. A supposedly 'quadratic' algorithm that divides by $2a$ without this check is not correct for the stated input domain.

10. Textbook Exercise - Levitin 1.2-Q8

Paraphrased task: give a problem, other than GCD, for which you know more than one algorithm. Compare which solution is simpler and which is more efficient.

Model answer using searching

Problem: find a target value in a collection.

Algorithm 1: Linear Search - examine values one by one.

Algorithm 2: Binary Search - repeatedly discard half of a sorted search interval.

Simplicity: Linear Search is usually simpler and works on unsorted data.

Efficiency: Binary Search can be much more efficient on sorted, random-access data.

Lecture 3 connection

Before asking which is faster, first establish that each algorithm is correct under its own preconditions. Binary Search, for example, depends on the precondition that the data is sorted. If that condition is violated, an efficiency comparison is meaningless because the algorithm may return the wrong answer.

11. Optional Textbook Bridge - CLRS 2.1-3

Paraphrased task: write a Linear Search algorithm for a sequence A and target v, returning an index if v occurs and NIL otherwise; then prove the algorithm correct using a loop invariant.

Why this is marked optional: the input/output specification and correctness goal fit Lecture 3 perfectly, but the full loop-invariant proof technique is the formal subject of Lecture 4.

Lecture 3 level specification

Precondition: A is a finite sequence and v is comparable for equality with its elements.

Postcondition: either an index i is returned with $A[i] = v$, or NIL is returned and v does not occur anywhere in A.

Termination: the algorithm checks at most n elements, so it must finish.

Preview of the invariant

At the start of iteration i, none of the already examined elements $A[1..i-1]$ equals v. Lecture 4 will formalize how initialization, maintenance, and termination use this statement to prove correctness.

12. Practice Set - Apply Lecture 3 Reasoning

#	Question	Model answer
P1	For SQRT(x), state a reasonable precondition if the algorithm is intended to return only real results.	$x \geq 0$.
P2	For FIRST(A), which returns A[0], what precondition is essential?	A must be non-empty.
P3	An algorithm returns $m = 100$ for $A = [3, 8, 5]$. Which MAX postcondition does it fail?	It fails.
P4	If an algorithm always returns the correct answer whenever it terminates, but may do so for ever, what has been established?	Partial correctness.
P5	Why is 'the output looks reasonable' not a valid postcondition?	It is vague and not mathematically/testably defined.
P6	If Binary Search is run on unsorted data, which part of the correctness specification has been violated?	Assumption that input is sorted.

Recommended student submission: Levitin 1.2-Q3 and 1.2-Q4, plus Practice P1-P6. Use CLRS 2.1-3 after Lecture 4 when loop invariants have been formally covered.

13. References and Edition Notes

Anany Levitin, Introduction to the Design and Analysis of Algorithms, 3rd ed., Chapter 1, especially §1.2 and Exercises 1.2-Q3, Q4, Q8.

Cormen, Leiserson, Rivest, Stein (CLRS), Introduction to Algorithms, 3rd ed., Exercise 2.1-3 for Linear Search correctness/loop-invariant practice.

Edition note: the CSC301 CDF associates Lecture 3 with CLRS Chapter 5. In CLRS 3rd edition, Chapter 5 is probabilistic analysis/randomized algorithms, so this handout does not use Chapter 5 questions for correctness. CLRS 2.1-3 is included only because it is a direct correctness exercise and is clearly marked as a bridge to Lecture 4.

Textbook exercise prompts in this handout are paraphrased rather than reproduced verbatim. Worked solutions are written for this course and emphasize the Lecture 3 correctness vocabulary.